

Volume 17

Sentinel Detection Framework (SDF)

Version: 1.0

Status: Draft

Vision

The Sentinel Detection Framework is the common architecture used by every security engine.

Instead of each module inventing its own detection logic, every engine follows the same lifecycle.

Whether we scan:

- Websites
- APIs
- Windows
- Linux
- macOS
- Android
- Cloud
- Containers
- Kubernetes
- Email
- Active Directory

...the same detection model is used.

Chapter 1

Detection Pipeline

Discovery

Every engine follows this pipeline.

Chapter 2

Detection Types

The framework supports multiple detection methods.

Configuration Analysis

Examples

- Missing HTTPS
 - Weak TLS
 - Missing CSP
 - Weak permissions
 - Open cloud bucket
 - Weak SSH configuration
-

Behavioral Analysis

Examples

- New startup process
 - Unusual network traffic
 - Unexpected administrator login
 - Service suddenly exposed to the internet
-

Integrity Analysis

Examples

- File changed
 - Configuration drift
 - Certificate replaced
 - Unauthorized policy modification
-

Vulnerability Analysis

Examples

- Known CVEs
 - Weak dependencies
 - Unsupported software
 - Outdated frameworks
-

Exposure Analysis

Examples

- Internet exposure

- Open ports
 - Public APIs
 - Anonymous access
 - Weak authentication
-

Identity Analysis

Examples

- Dormant accounts
 - Missing MFA
 - Excessive privileges
 - Weak authentication policies
-

Chapter 3

Detection Evidence

Every detection requires evidence.

Example

Finding

The evidence is stored as part of the finding.

Chapter 4

Confidence Engine

Every finding receives a confidence rating.

Factors may include:

- Number of independent observations.
- Reliability of the source.
- Freshness of the data.
- Consistency with previous observations.
- Correlation with other findings.

The confidence calculation is documented and explainable.

Chapter 5

Detection Severity

Project Sentinel separates three concepts:

Severity – How serious is the issue?

Likelihood – How likely is exploitation or misuse?

Business Impact – What is the effect on the organization?

These values combine into a transparent prioritization model.

Chapter 6

Detection Rules

Every rule contains:

- Rule ID
- Name
- Description
- Scope
- Required Evidence
- Detection Logic
- Severity
- Confidence Logic
- Remediation Guidance
- References
- Test Cases

Rules are versioned and documented.

Chapter 7

Rule Packs

Rules are grouped into packs.

Examples

- Web Security Pack
- Mobile Security Pack
- Windows Security Pack
- Linux Security Pack
- Cloud Security Pack
- Kubernetes Pack
- Identity Pack
- Email Security Pack
- API Security Pack

Organizations can enable only the packs they need.

Chapter 8

Verification

After remediation:

The same rule executes again.

Example

Finding

Verification is mandatory before a finding is considered resolved.

Chapter 9

False Positive Management

Users can mark a finding as:

- Confirmed
- False Positive
- Accepted Risk
- Under Investigation

The decision becomes part of the audit history.

Chapter 10

Detection Explainability

Every finding answers:

- What was observed?
 - Why is it important?
 - What evidence supports it?
 - How confident is the result?
 - What should be done?
 - How can the fix be verified?
-

PS-ADR-0014

Unified Detection Architecture

Status: Accepted

Decision

All Project Sentinel security engines shall implement the Sentinel Detection Framework.

Rationale

A unified detection framework improves consistency, testability, maintainability, and user experience.

NEW CORE ENGINE

Detection Orchestrator

The Detection Orchestrator coordinates all detection activities.

Responsibilities include:

- Selecting rule packs.
 - Scheduling analyses.
 - Managing worker processes.
 - Preventing duplicate work.
 - Coordinating evidence collection.
 - Reporting progress.
 - Triggering verification.
-

NEW CORE ENGINE

Rule Validation Engine

Every detection rule is automatically tested before release.

Validation includes:

- Syntax.
- Logic.
- Performance.
- Expected evidence.
- Regression tests.
- Documentation completeness.

This ensures high-quality rule sets.

NEW CORE ENGINE

Remediation Knowledge Engine

Every finding links to structured remediation guidance.

Each remediation includes:

- Plain-language explanation.
- Technical steps.
- Estimated time.

- Estimated risk reduction.
- Rollback guidance.
- Verification procedure.
- References to official vendor documentation where applicable.

The engine supports both beginner and expert audiences.

NEW IDEA

Cyber Health Index (CHI)

Instead of a single security score, Project Sentinel displays multiple health dimensions:

Dimension	Example
-----------	---------

Endpoint Health	96%
-----------------	-----

NEW IDEA

Security Confidence Dashboard

A dashboard focused on **confidence**, not fear.

Example:

Environment Coverage

The dashboard emphasizes progress, evidence, and priorities instead of overwhelming users with alerts.

A Unique Vision: The Sentinel Principles

I propose we formally define the engineering philosophy that every contributor must follow.

The Ten Sentinel Principles

1. **Evidence Before Conclusions** – Never claim more than the evidence supports.
2. **Simple by Default** – Powerful features should not require a complex interface.
3. **Mission First** – Users think in goals, not modules.
4. **Everything Is an Object** – A consistent object model across the platform.
5. **Security by Design** – Security is built into every layer.

6. **Privacy by Design** – User data belongs to the user.
 7. **Accessibility by Design** – Target WCAG 2.2 AA across the platform.
 8. **Open Architecture** – Use open standards, documented APIs, and open-source components wherever practical.
 9. **Verify Everything** – Remediation is not complete until verified.
 10. **Learn Continuously** – Every mission improves the platform's understanding of the authorized environment.
-

Strategic Vision Beyond Version 1

Looking ahead, I can see Project Sentinel eventually growing into a complete ecosystem with:

- **Sentinel Core** – The Cyber Security Operating Platform.
- **Sentinel Home** – Simplified protection for individuals and families.
- **Sentinel Professional** – For consultants and authorized bug bounty researchers.
- **Sentinel Enterprise** – Large organizations with advanced governance.
- **Sentinel Government** – High-assurance deployments with offline operation and enhanced policy controls.
- **Sentinel SDK** – For third-party modules, automation, and integrations.
- **Sentinel Academy** – Interactive training, labs, and guided learning built directly into the platform.

All of these editions would share **one codebase, one object model, one plugin architecture**, and **one engineering philosophy**. That keeps the project maintainable while allowing it to scale from a single laptop to a multinational organization.

I believe this is a realistic long-term direction. It is ambitious, but every capability we are adding is grounded in technologies that can be implemented incrementally with Python, open-source libraries, Linux, and well-defined interfaces. If we maintain this discipline, Project Sentinel can become not just another security product, but a lasting cybersecurity engineering platform.

====[USER]=====

Good and continue

====[ASSISTANT]=====

Excellent. I think we are now approaching what will become the **technical DNA** of Project Sentinel.

From now on, we should begin designing the platform exactly like companies such as Microsoft, Google, VMware, Palo Alto, and CrowdStrike design their products—not by thinking about features first, but by designing reusable platform services.

That is how we build something that can still evolve in 15 years.

PROJECT SENTINEL BLUEPRINT

Volume 18